PS 2

Muchang Bahng

Fall 2026

1 Problem 1

All code is in `p1.py`. Numbers below are from `results/p1.json`.

Exercise 1.1

So far in this course we have focused on fitting models $f_\mu(x)$ to approximate $\mathbb{E}[Y|X]$, the conditional mean of the target Y given the input X. Suppose we want to also incorporate a measure of uncertainty in our observations of Y, in the form of a function $f_\sigma(x) = \text{Var}(Y|X = x)$ (the conditional standard deviation) which models how the observation noise changes with the input X.

(a) Say we have a neural architecture $g : \mathbb{R}^p \to \mathbb{R}^2$ that takes in a p-dimensional feature vector and returns a 2-dimensional vector of real numbers. Suggest a link function that would be appropriate for converting the output to f_μ and f_σ.

(b) We could model data of this form as $Y|X \sim \mathcal{N}(f_\mu(X), f_\sigma(X)^2)$. Write down a loss function we could use for model training based on this model.

(c) We have provided a dataset from taxi trips taken in NYC in `taxi_trips.parquet`. It contains 8 total columns, 7 features and 1 target (trip duration, in minutes). The 7 features (pickup/dropoff coordinates and time-of-day, day-of-week and day-of-year) have already been standardized, so do not expect them to look like raw latitudes and longitudes. Define a multi-layer perceptron in PyTorch or JAX that outputs a conditional mean and standard deviation, and fit this on the taxi trip dataset. Make sure to follow best practices: define a baseline (or multiple) baseline models that you think your model should beat, split your dataset into training/validation/test sets, etc.

(d) How would we use the model we've just fit to predict the probability that a trip will take less than 45 minutes? The normal distribution assumption is fine.

(e) Estimate these probabilities on your held-out test set and evaluate them. Report both a proper scoring rule for the predicted probabilities (e.g. the Brier score or the log-loss) and the classification accuracy obtained by thresholding the probability at 0.5.

Solution. **(a)** Take $g(x) = (g_1, g_2)$ and set $f_\mu(x) = g_1(x)$ (identity link, since the mean is unconstrained) and $f_\sigma(x) = \text{softplus}(g_2(x)) + \epsilon = \log(1 + e^{g_2(x)}) + \epsilon$ with $\epsilon = 10^{-3}$.
(b) With $Y|X \sim \mathcal{N}(f_\mu(X), f_\sigma(X)^2)$ and i.i.d. samples $(x_i, y_i)_{i=1}^n$, the likelihood is

$$L(\theta) = \prod_{i=1}^{n} \frac{1}{\sqrt{2\pi}\, f_\sigma(x_i)} \exp\left(-\frac{(y_i - f_\mu(x_i))^2}{2 f_\sigma(x_i)^2} \right).$$

Taking the log turns the product into a sum:

$$\begin{aligned} \log L(\theta) &= \sum_{i=1}^{n} \left[-\log\big(\sqrt{2\pi}\, f_\sigma(x_i)\big) - \frac{(y_i - f_\mu(x_i))^2}{2 f_\sigma(x_i)^2} \right] \\ &= -\sum_{i=1}^{n} \left[\log f_\sigma(x_i) + \frac{(y_i - f_\mu(x_i))^2}{2 f_\sigma(x_i)^2} \right] - \frac{n}{2} \log 2\pi. \end{aligned}$$

Maximizing the likelihood is equivalent to minimizing the negative log-likelihood (dropping the constant), which we use as the loss:

$$\mathcal{L}(\theta) = \frac{1}{n}\sum_{i=1}^{n}\left[\log f_\sigma(x_i) + \frac{(y_i - f_\mu(x_i))^2}{2f_\sigma(x_i)^2}\right].$$

(c) The data (523,554 trips) were split randomly 70/15/15 into train/val/test (366,487/78,533/78,534). The model is an MLP $7 \to 128 \to 128 \to 128 \to 2$ with ReLU, trained with Adam (lr 10^{-3}, batch 1024), with early stopping on validation NLL (patience 8). Baselines: (B1) a global Gaussian with the training mean and std; (B2) linear regression for the mean with a constant σ (the training residual std); (B3) a linear mean $f_\mu(x) = w^\top x + b$ and a linear-plus-softplus standard deviation $f_\sigma(x) = \text{softplus}(v^\top x + c) + \epsilon$, i.e. the same link and NLL loss as the MLP but with g linear, trained with the same optimizer settings. Test results:

Model	test NLL	test RMSE (min)
B1 constant Gaussian	3.466	7.746
B2 linear mean, constant σ	3.441	7.551
B3 linear mean, softplus-linear σ	3.440	7.769
MLP (mean, σ)	**2.680**	**3.806**

The MLP beats all three baselines by a wide margin; the linear models (B2, B3) are barely better than a constant, and letting σ depend linearly on the inputs (B3) adds almost nothing (NLL 3.440 vs. 3.441; B3 had not met the early-stopping criterion after 100 epochs, which likely explains its slightly higher RMSE than OLS), so the signal is strongly nonlinear in the coordinates. The predicted σ varies from 0.75 to 31.8 (median 3.3), i.e. the model does learn heteroskedasticity: the predicted σ is strongly positively correlated with the predicted mean ($\rho = 0.79$ on the test set), so longer trips are predicted to be noisier.

(d) Since $Y|X = x \sim \mathcal{N}(f_\mu(x), f_\sigma(x)^2)$,

$$\mathbb{P}(Y < 45 \mid x) = \Phi\Big(\frac{45 - f_\mu(x)}{f_\sigma(x)}\Big),$$

where Φ is the standard normal CDF.

(e) On the test set 99.55% of trips are shorter than 45 minutes, so the problem is extremely imbalanced.

Predictor	Brier	log-loss	accuracy@0.5
MLP	**0.00306**	**0.0208**	**0.9963**
B1 constant Gaussian	0.00446	0.0552	0.9955
B2 linear + const. σ	0.00446	0.0466	0.9955
B3 linear + softplus σ	0.00446	0.0478	0.9955
Constant empirical rate	0.00444	0.0286	0.9955

The MLP improves the Brier score by about 31% and log-loss by 27% over the best constant predictor. Accuracy is almost uninformative here: always predicting "< 45" already gives 99.55%, and the MLP only reaches 99.63%. Proper scoring rules are the meaningful metric. Caveat: the Gaussian is a poor fit for a positive, right-skewed target, so the tail probabilities are only approximate (the log-loss of the constant Gaussian is worse than the constant empirical rate, showing this).

2 Problem 2

All code is in `p2.py`; results in `results/p2_seed*.json` (run as `python p2.py <seed>`).

Exercise 2.1

For this problem, you will explore different methods for encoding categorical random variables. We have provided a dataset `retail.csv`, which contains data on retail transactions. We've cleaned up the

data a bit so you only have two features: `StockCode` and `CustomerID`, both of which are categorical. (You may need to convert them to categorical data types in your code for the encoding methods to work properly.) The target is `Quantity`, which is a positive integer. Note that `Quantity` is very heavy-tailed (most values are small, but a few are enormous), so think about what loss function and/or target transformation is appropriate.

(a) Start by exploring the cardinality of the two features. How many unique values are there for each feature? What does this suggest about how you might encode them? Is one-hot encoding feasible? To prepare for the next steps, split the data into training, validation, and test sets. The validation set will allow you to use early stopping during training for the neural network models.

(b) Try target encoding. For each unique value of the feature, replace it with the mean of the target for that value (from the training set). You can use `TargetEncoder` from the `category_encoders` package. Using an appropriate loss function, fit a model of your choice to the data using the target-encoded features. Evaluate the model on the test set.

(c) Try a neural network model with an embedding layer for each of the two features. Choose a reasonable embedding dimension. Fit this model to the data. Evaluate the model on the test set. How does it compare to the target encoding model?

(d) What if we combine the two? That is, use both the target encoded features and the embeddings as input to the neural network. Fit this model to the data. Evaluate the model on the test set. How does it compare to the other two models?

(e) Try to give some intuition for your results. What is the embedding layer doing? Why might it be useful in this context? Evaluate your findings on the test set.

Solution. **(a)** There are 397,924 rows, with **3,665** unique `StockCode`s and **4,339** unique `CustomerID`s. One-hot encoding would need $3665 + 4339 = 8004$ columns. That is technically feasible with sparse matrices, but it is wasteful and high-dimensional, most levels appear rarely so each column is learned from very little data, and it gives no way for related levels to share information. Target encoding (one number per feature) and learned low-dimensional embeddings are more natural. `Quantity` is heavy-tailed (median 6, mean 13.0, max 80,995), so I model $\log(1 + \text{Quantity})$ and train with the Huber loss ($\delta = 1$) on this scale, so that the few giant orders do not dominate the fit. The data are split at random 70/15/15: 278,546 train, 59,689 validation, and 59,689 test rows. Levels are mapped to integer codes using the training set only. 0.12% of test rows contain at least one level not seen in training.

Protocol and metrics. All three neural models use the same MLP head ($64 \to 64 \to 1$, ReLU, dropout 0.1), AdamW (lr 3×10^{-3}, weight decay 10^{-4}, batch 512) and early stopping on validation MSE (patience 6). Because differences between models are small, I repeated each fit with 3 random seeds (same data split) and report mean $\pm$ std over seeds on the test set. Metrics: RMSE and MAE on the log scale, and MAE / median absolute error on the original quantity scale (prediction = expm1 of the log prediction).

Model	RMSE (log)	MAE (log)	MAE (qty)	MedAE (qty)
Predict train mean	1.001	0.815	10.37	4.19
(b) Target encoding + MLP	0.592 ± 0.003	0.427 ± 0.003	6.88 ± 0.03	1.75 ± 0.02
(c) Embeddings + MLP	$\mathbf{0.527 \pm 0.001}$	0.362 ± 0.001	6.02 ± 0.05	1.38 ± 0.02
(d) Both	0.530 ± 0.003	$\mathbf{0.361 \pm 0.004}$	$\mathbf{5.96 \pm 0.06}$	$\mathbf{1.31 \pm 0.01}$

(b) `TargetEncoder` (smoothing 10) is fit on the training set only and maps each of the two features to a smoothed mean of $\log(1 + Q)$; levels unseen in training receive the global mean. An MLP on these two numbers gets test RMSE 0.592, a large improvement over the constant baseline (1.001).

(c) Each feature gets a 16-dimensional embedding (index 0 is reserved for unseen levels); the two embeddings are concatenated and fed to the same MLP. Test RMSE is 0.527, about 11% lower than target encoding, and the gap (0.065) is more than 20 times the seed-to-seed standard deviation.

(d) Feeding the target-encoded values together with the embeddings gives RMSE 0.530, MAE (log) 0.361, and slightly better quantity-scale errors (MAE 5.96, MedAE 1.31). It is clearly better than target encoding alone, but *indistinguishable* from embeddings alone: the RMSE difference (0.003) is about the size of the seed noise, and the other metrics differ in the opposite direction by a similar small

amount. So adding target encoding on top of embeddings brings no meaningful gain.

(e) *Intuition.* Target encoding compresses each level to a single scalar (its average target), so the network sees only two numbers per row, "how large is this item typically" and "how large does this customer typically order", and cannot represent anything else about the levels. An embedding layer instead learns a vector for each level, trained end to end on the loss, so levels are placed in a latent space where items (and customers) that behave similarly end up close. The MLP can then model interactions between the item and customer vectors (for example, a wholesale customer buying a cheap item), and a level's vector is informed by every row in which it appears. In particular, the embedding model can recover the information in the target encoding: one mean-like direction is easy to learn. As a sanity check, the first principal component of the learned `StockCode` embeddings (seed 0) has correlation -0.58 with the per-item mean log-quantity (the sign of a principal component is arbitrary), so the embedding space is organized in part by the same quantity target encoding uses, and it carries more dimensions on top of that. This is consistent with (c) and (d): embeddings outperform target encoding by about 11% in RMSE, and adding target encoding to embeddings brings nothing.

A check of one possible explanation. A natural worry is that the target encoding of a training row includes that row's own label (leakage), so a network that also sees it might over-trust it. I tested this by replacing the training-set encoding with a 5-fold out-of-fold encoding (validation and test still use the encoder fit on all of train). It changed nothing: target-encoding-only RMSE is 0.593 ± 0.005 (vs. 0.592) and combined RMSE is 0.526 ± 0.003 (vs. 0.530), the latter within noise of embeddings alone. So with this smoothing and data size leakage does not explain anything here; the conclusion is simply that the embeddings already capture what the target encoding offers.

Unseen levels. For the 73 test rows with an unseen level (0.12%), all models have much larger errors than on the other rows: log-RMSE is about 1.04–1.26 (across models and seeds) versus about 0.52–0.60 for rows with all levels seen. They still beat the mean baseline on those same rows (RMSE 1.39), but with only 73 rows the differences between models there are not reliable. These levels have no learned representation, so a fallback (global mean or cold-start features) is all one can do.

3 Problem 3

All code is in `models.py` (RKS and MLP implementations) and `p3.py` (sweeps, predictions, figures). The data have 600 train, 200 validation, and 200 test points.

Exercise 3.1

In class, we have discussed an important idea in machine learning: constructing a large set of features and then fitting relatively simple models—such as linear models or linear models composed with simple nonlinear transformations—on top of these features. In particular, we have seen how both the Random Kitchen Sink (RKS) model and the Multi-Layer Perceptron (MLP) fall under this general paradigm. In this problem, you will explore, implement, and evaluate these two models on a toy dataset.

A brief reminder on Random Kitchen Sink: Given covariates $x_i \in \mathbb{R}^d$ and responses $y_i \in \mathbb{R}$, the RKS model first maps each covariate into a higher-dimensional feature space using randomly generated sinusoidal features. Specifically, consider h pairs of feature functions $(\phi_j^{\sin}, \phi_j^{\cos})_{j=1}^h$, defined as

$$\phi_j^{\sin}(x_i) = \sin(w_j^T x_i + b_j), \qquad \phi_j^{\cos}(x_i) = \cos(w_j^T x_i + b_j),$$

where $b_j \sim U[0, 2\pi]$ and $w_j \sim \mathcal{N}_d(0_d, I_d/\sigma^2)$, with σ^2 controlling the scaling of the random projection. The resulting feature representation is

$$z_i = \left[\phi_1^{\sin}(x_i), \phi_1^{\cos}(x_i), \ldots, \phi_h^{\sin}(x_i), \phi_h^{\cos}(x_i)\right]^\top \in \mathbb{R}^{2h}.$$

A simple linear model is then fit on these features to predict the response. Because the number of features $2h$ can be large relative to the number of samples, we employ ridge regression (i.e., linear regression with L^2 regularization) to prevent overfitting. (In the provided starter code the total number of random features $2h$ is called D.)

(a) **Data visualization:** You are given a pre-generated dataset with training, validation, and testing splits. Begin by visualizing the training data. The dataset corresponds to samples (x_1, x_2, y) and you can be assured that (x_1, x_2) are each bounded between $[-3, 3]$. You can either generate a 2D grid-plot where the y value is represented by a color at different (x_1, x_2) locations, or a 3D plot where we visualize the response y on the z-axis.

(b) **Random Kitchen Sink with Ridge Regression:** Implement and fit an RKS model to the given dataset with ridge regression. You will need to vary one of the key hyperparameters while keeping others fixed at reasonable defaults. Possible hyperparameters include: the number of random features h, the kernel scale parameter σ, or the ridge regularization strength λ. Perform a hyperparameter sweep over your chosen parameter and record both the training and validation losses (MSE). Plot the losses as a function of the hyperparameter (two plots, one for training and one for validation loss), and select the best hyper-param based on the validation loss.

(c) **1-Hidden Layer Multi-Layer Perceptron:** Next, fit the same dataset using a one-hidden-layer MLP implemented in PyTorch or Jax (the provided starter code is in PyTorch). As before, you will choose one hyperparameter to vary (for example, the hidden layer dimension, learning rate, or batch size) and keep other hyperparameters fixed at default values. Plot the training and validation losses over the sweep and determine the best-performing configuration based on the validation loss.

(d) **Comparison and visualization:** Using the best-performing hyperparameters from the previous steps, refit both models and report the validation loss (MSE) of each. Discuss which approach performs better and why. The provided `test.csv` is unlabeled (it contains only x_1, x_2): produce predictions from both models on it and submit them as two CSV files, `rks_test_predictions.csv` and `mlp_test_predictions.csv`, one prediction per row in the same order as `test.csv`. Finally, visualize the fitted functions from the two models over the input space (we provide an example function to do this). Compare these surfaces with your visualization of the training data from part (a), and qualitatively explain how each model captures the underlying structure of the data.

Provided resources: `train.csv/val.csv/test.csv` (the train/validation/test data points; `test.csv` has no y column), `dataloader.py` (a handy data structure to load the given data), `plotting.py` (an example plotting function that helps with the visualization in part (d)), and `models.py` (skeleton code for implementing the RKS/MLP methods, performing training as well as hyper-parameter sweeping). Note that you should feel free to implement your own code (i.e. ignore these helper files) as long as it's in Python and you are using the provided dataset. Make sure to attach your code as part of the assignment submission.

Solution. **(a)** Figure 1 shows the training data. The response looks like a smooth surface: low (about -1) around $(x_1, x_2) \approx (-1.5, 0)$, rising toward high values near $x_2 = \pm 3$, especially at $x_1 \approx 1.5$, with a saddle around $x_1 \approx 1.5,\ x_2 = 0$.

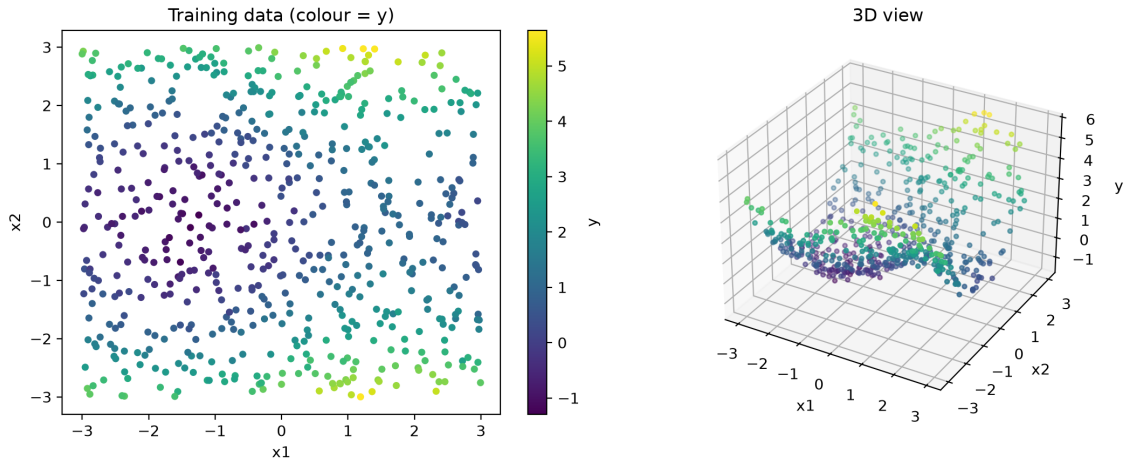


Figure 1: Training data.

(b) RKS with $D = 2000$, $\lambda = 10^{-3}$ fixed, sweeping σ (the smoothness; $w \sim N(0, I/\sigma^2)$). Small σ means high-frequency features: heavily overfit (val MSE 0.98 at $\sigma = 0.1$; at $\sigma = 0.2$ the validation MSE is about 20; this is not a seed fluke, over 5 random feature draws it ranged from 14 to 44, i.e. with $2000 > 600$ very high-frequency features and tiny λ the fit essentially interpolates the noise and extrapolates wildly between training points). Validation MSE decreases to a minimum of 0.0362 at $\sigma = 6$ (Figure 2); the best $\sigma = 6$. Sweeping D with $\sigma = 6$ shows MSE flattening beyond $D \approx 50$–100 (0.067 at $D = 20$, 0.036 at $D = 1000$), best $D = 1000$ (the values for $D \geq 500$ differ only in the fourth decimal). A λ sweep at $\sigma = 6$ (`figs/p3_rks_lambda.png`) is flat between 10^{-6} and 1 (0.036–0.037), so I kept $\lambda = 10^{-3}$. Over 5 random feature draws the final configuration has validation MSE 0.0360–0.0364.

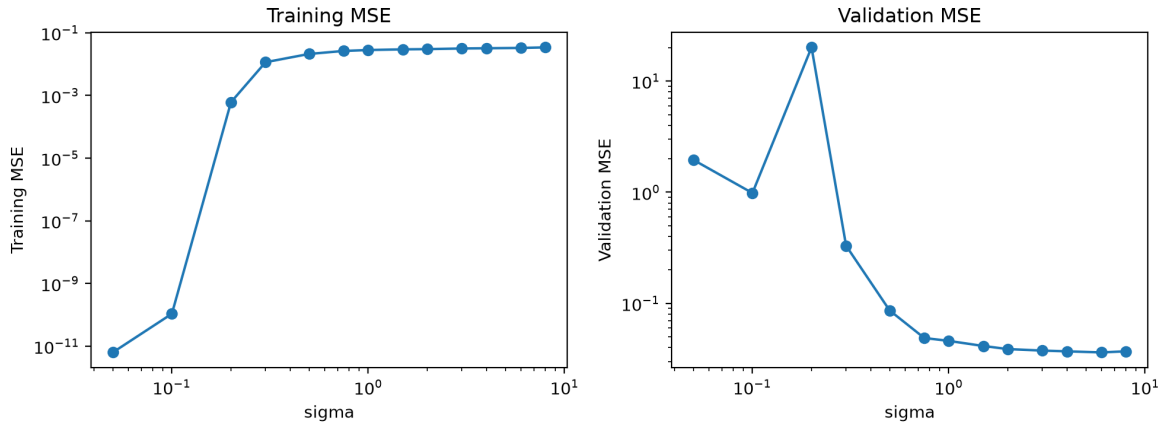


Figure 2: RKS: train and validation MSE vs. σ.

(c) One-hidden-layer ReLU MLP, Adam, lr 10^{-3}, batch 128, 300 epochs, sweeping hidden width $\{2, \ldots, 512\}$. Validation MSE falls from 0.37 (width 2; width 4 was an unlucky optimization run at 0.94: with other random initializations it gives 0.34 and 0.38) to 0.0390 at width 512, which is the best (Figure 3). The curve flattens beyond 256 (0.0397), so the larger widths mostly buy small gains.

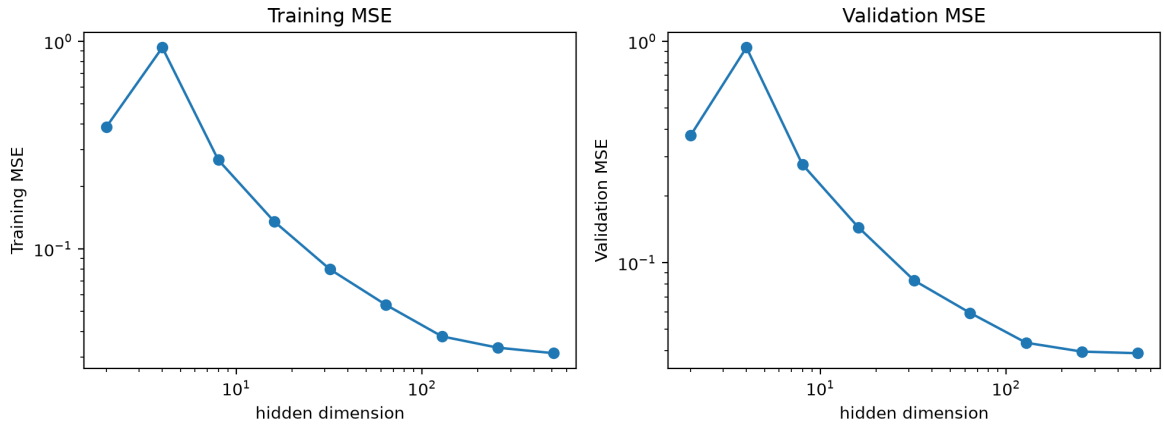


Figure 3: MLP: train and validation MSE vs. hidden width.

(d) Refitting with the best hyperparameters:

Model	train MSE	validation MSE
RKS ($D = 1000, \sigma = 6, \lambda = 10^{-3}$)	0.0330	**0.0360**
MLP (width 512)	0.0314	0.0390

(Variance of y on validation is 2.40, so both models explain over 98% of it.) RKS is better on validation, and it is also much cheaper (a closed-form linear solve vs. 300 epochs of SGD). The difference (0.036 vs. 0.039) is small in absolute terms and the validation set has only 200 points (and was also used to pick the hyperparameters, so these numbers are slightly optimistic). The ordering is, however, consistent across random seeds: RKS 0.0360–0.0364 over 5 feature draws, MLP 0.0376–0.0390 over 3 initializations. So RKS is modestly but reliably better on this problem, but I would not call it a large effect. RKS, with large σ, builds in a smoothness prior that matches this smooth target, and its only trained parameters are linear, so it is a convex problem with no optimization noise. The MLP has to learn its features, is a bit more overfit (larger train–val gap), and with ReLU units its surface is piecewise linear, which is a slightly worse match to a smooth function with 600 points. Predictions on `test.csv` are in `rks_test_predictions.csv` and `mlp_test_predictions.csv` (one per row, same order).

Figure 4 shows the fitted surfaces. Both recover the structure seen in (a): the minimum near $(-1.5, 0)$, the rise toward $x_2 = \pm 3$ with maxima at $x_1 \approx 1.5$, and the saddle near $(1.5, 0)$. The RKS surface is very smooth with nearly elliptical level sets, as expected from a sum of sinusoids at a large length scale. The MLP surface has the same global shape but its contours are slightly wobbly and polygonal, reflecting the piecewise-linear ReLU regions, consistent with a slightly less smooth fit.

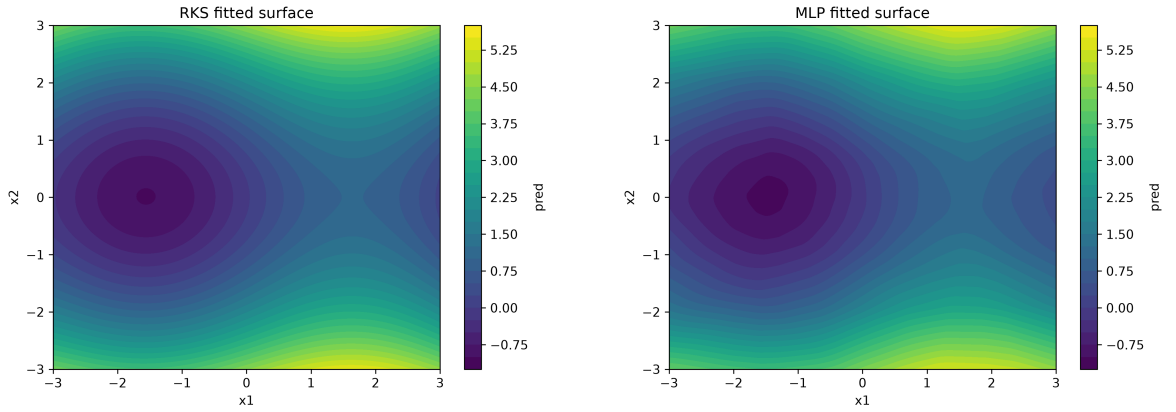


Figure 4: Fitted surfaces: RKS (left) and MLP (right).